

MoonBit 国产基础软件生态开源大赛 2026

项目申报书

一、参赛方向

基础数据结构与算法

二、项目名称

indexmap — 插入顺序保持哈希映射

三、项目简介

indexmap 是一个为 MoonBit 语言实现的插入顺序保持哈希映射（Insertion-Order-Preserving Hash Map）。它结合了哈希表的 $O(1)$ 平均查找效率和数组的顺序访问能力，同时保证迭代时元素始终保持插入顺序。

四、项目特色

4.1 核心功能

- $O(1)$ 平均时间复杂度的键值查找
- $O(1)$ 时间复杂度的插入顺序索引访问
- 迭代始终按插入顺序进行
- Swap-remove 语义：删除操作使用交换-移除策略，保持内部数组紧凑

4.2 数据结构设计

采用以下核心设计：

- 开放寻址哈希表：使用线性探测解决冲突，与 MoonBit 标准库 HashMap 一致
- 分离的顺序数组：单独维护插入顺序，与哈希表解耦
- 键存储策略：顺序数组存储键而非索引，使 rehash 操作不影响顺序

4.3 实现细节

```
struct Slot[K, V] {
    hash : Int
    key : K
    mut value : V
}

struct IndexMap[K, V] {
    mut entries : FixedArray[Slot[K, V]?]
    order : Array[K]
    mut capacity : Int
    mut capacity_mask : Int
    mut size : Int
}
```

- 哈希表：使用 `FixedArray[Slot[K, V]?]` 存储键值对
- 顺序数组：使用 `Array[K]` 按插入顺序存储键
- 容量管理：自动扩容，负载因子为 0.5

五、API 设计

5.1 构造函数

函数	说明
<code>IndexMap::new()</code>	创建空映射
<code>IndexMap::with_capacity(n)</code>	创建至少 <code>n</code> 个槽位的映射
<code>IndexMap::from(arr)</code>	从数组创建映射

5.2 核心操作

方法	说明
<code>m.set(key, value)</code>	插入或更新
<code>m.get(key) -> V?</code>	查找，返回 <code>Option</code>
<code>m.at(key) -> V</code>	查找，缺失时 <code>panic</code>
<code>m.contains_key(key)</code>	键存在性检查
<code>m.remove(key) -> V?</code>	删除并返回值

5.3 索引访问

方法	说明
<code>m.get_index(i)</code>	位置 <code>i</code> 的键值对
<code>m.get_index_key(i)</code>	位置 <code>i</code> 的键
<code>m.get_index_value(i)</code>	位置 <code>i</code> 的值

5.4 迭代器

方法	说明
<code>m.iter()</code>	按顺序遍历键值对
<code>m.keys_iter()</code>	按顺序遍历键
<code>m.values_iter()</code>	按顺序遍历值

5.5 运算符

- `m[key]` — 括号读取（缺失时 `panic`）
- `m[key] = value` — 括号写入

六、特性实现

6.1 Eq 特性

支持两个 `IndexMap` 的相等性比较。由于使用哈希表实现，比较是无序的（不同插入顺序但相同内容的映射相等）。

6.2 Debug 特性

支持调试输出，显示为 `IndexMap({key: value, ...})` 格式。

七、测试覆盖

项目包含 30 个测试用例，覆盖：

- 基本 CRUD 操作（创建、读取、更新、删除）
- 括号运算符（读取和写入）
- 插入顺序保持
- Swap-remove 语义
- 自动扩容（100 个条目）
- 大规模数据正确性（1000 个条目）
- 字符串和整数键
- 重复键处理
- Eq 特性
- 空映射迭代

测试结果：共 30 个测试，全部通过，0 个失败。

八、使用示例

```
test {  
  // 创建并填充  
  let m = @indexmap.IndexMap::from([("b", 2), ("a", 1), ("c", 3)])  
  
  // 键查找 — O(1)  
  assert_eq(m.get("a"), Some(1))  
  assert_eq(m["b"], 2) // 缺失时 panic  
  
  // 迭代保持插入顺序  
  let keys = m.keys_iter().to_array() // ["b", "a", "c"]  
  
  // 索引访问  
  assert_eq(m.get_index_key(0), Some("b"))  
  assert_eq(m.get_index(1), Some(("a", 1)))  
  
  // 更新（保持原始位置）  
  m.set("a", 99)  
  // 仍然: ["b", "a", "c"], 但 m["a"] == 99  
  
  // 删除  
  assert_eq(m.remove("a"), Some(99))  
  // 现在: ["b", "c"]  
}
```

九、安装方式

```
moon add moonbit-community/indexmap
```

然后在 .mbt 文件中导入：

```
let m = @indexmap.IndexMap::new()
```

十、项目结构

```
indexmap/
.github/workflows/ci.yml # GitHub Actions CI
indexmap.mbt             # 主实现文件
indexmap_test.mbt        # 测试文件
moon.mod.json            # 模块元数据
moon.pkg                 # 包依赖
README.md                # 英文文档
DECLARATION.md           # 项目申报书
DECLARATION.pdf          # 项目申报书 PDF
LICENSE                  # Apache-2.0 许可证
```

十一、技术亮点

- 与标准库一致的设计：采用与 MoonBit 标准库 HashMap 相同的开放寻址 + 线性探测策略
- 高效的内存布局：使用 FixedArray 存储哈希表，内存连续
- 智能扩容策略：负载因子 0.5，平衡空间和性能
- 类型安全：完整的泛型支持，编译时类型检查
- MoonBit 惯用语法：使用 #alias 实现运算符重载，guard 模式匹配等

十二、未来扩展

- Entry API (entry(key).or_insert(default))
- 有序迭代（按排序顺序）
- 序列化/反序列化支持
- 并发安全版本
- 更多集合操作 (union, intersection, difference)

十三、许可证

Apache-2.0

十四、联系方式

- 项目地址：<https://gitlink.org.cn/yangayang/indexmap>
- 问题反馈：GitLink Issues

本项目参加 MoonBit 国产基础软件生态开源大赛 2026

方向：基础数据结构与算法